

1. Migraciones

Su misión: Crear las columnas físicas en la base de datos SQL. **El esquema mental:**

- **Estructura base:** Tiene un up (para crear) y un down (para borrar si te equivocas).
- **Los imprescindibles:** Siempre llevan un id (primaryKey, autoIncrement) y al final createdAt y updatedAt.
- **Las propiedades:** Tienes que definir el tipo (type: Sequelize.STRING, Sequelize.INTEGER, Sequelize.BOOLEAN, Sequelize.DOUBLE, Sequelize.TIME).
- **¡LA TRAMPA! (Claves foráneas y Nulos):**
 - ¿Es obligatorio el campo? allowNull: false. ¿Es opcional? allowNull: true.
 - ¿Se relaciona con otra tabla? Tienes que poner el bloque references: { model: { tableName: 'NombresEnPlural' }, key: 'id' }.
 - ¿Qué pasa si borran al padre? onDelete: 'CASCADE' (se borra también esto) o onDelete: 'SET NULL' (se queda vivo pero huérfano).

2. Modelos

Su misión: Es la representación en JavaScript de esa tabla de SQL y cómo habla con las demás. **El esquema mental:**

- **Estructura base:** Una clase que hace init() definiendo las propiedades (igual que la migración pero más resumido, ej. DataTypes.STRING).
- **Las Asociaciones (static associate):** ¡Aquí te juegas el examen!
 - Si **este** modelo tiene la clave foránea (ej. userId), usas belongsTo: `MiModelo.belongsTo(models.User, { foreignKey: 'userId', as: 'user' })`
 - Si **el otro** modelo tiene la clave foránea, usas hasMany (si tiene varios) o hasOne (si tiene uno): `User.hasMany(models.MiModelo, { foreignKey: 'userId', as: 'misModelos' })`

3. Validaciones

Su misión: El portero de discoteca. Comprueba que el req.body (el formulario) no traiga basura antes de molestar a la base de datos. **El esquema mental:**

- **Estructura base:** Son arrays de comprobaciones (const create = [...]).
- **Reglas básicas:** `check('campo').exists().isString().isLength({min: 1})`.
- **Reglas opcionales:** Si el campo no es obligatorio, usa `.optional({ nullable: true, checkFalsy: true })`.
- **Reglas customizadas:** Para comprobar lógicas complejas de base de datos (ej. "que este ID exista"), se usa `.custom(async (value, { req }) => { ... })`.

4. Middlewares

Su misión: Comprobar si tienes permiso para hacer lo que quieres hacer (Normalmente para Editar o Borrar). **El esquema mental:**

- **Estructura base:** `const checkOwnership = async (req, res, next) => { try/catch }.`
- **La lógica vital:**
 1. Buscas el elemento por el ID de la URL (`req.params.id`).
 2. Compruebas si el `userId` de ese elemento coincide con tu usuario (`req.user.id`).
 3. Si coincide: `return next()` (le dejas pasar al controlador).
 4. Si NO coincide: `return res.status(403).send('No tienes permiso')`.

5. Rutas

Su misión: El mapa de carreteras. Conecta una URL y un verbo HTTP con las funciones. **El esquema mental:**

- **Estructura base:** `app.route('/entidad/:id').verbo( ...lista_de_funciones... )`
- **EL ORDEN ESTRICTO (¡Súper importante!):**
 1. ¿Quién entra? (`isLoggedIn`, `hasRole('owner' | 'customer')`).
 2. ¿El elemento de la URL existe? (`checkEntityExists(Modelo, 'id')`).
 3. ¿Tiene permiso para tocarlo? (`TuMiddleware.checkOwnership`).
 4. ¿El formulario es válido? (`TuValidation.create`, seguido OBLIGATORIAMENTE de `handleValidation`).
 5. Ejecuta la acción (`TuController.create`).

6. Controllers

Su misión: El cerebro. Pide cosas a la base de datos y le devuelve el JSON al frontend. **El esquema mental:**

- **Estructura base:** `const accion = async function (req, res) { try/catch }`
- **Los imprescindibles:** ¡Siempre `async` y `try/catch`! En el `catch` siempre va `res.status(500).send(err)`.
- **Operaciones clave de Sequelize:**
 - **GET (Listar):** `await Modelo.findAll({ where: { ... } })`
 - **GET (Detalle):** `await Modelo.findByPk(req.params.id)`
 - **POST (Crear):** `const nuevo = Modelo.build(req.body); nuevo.userId = req.user.id; await nuevo.save(); res.json(nuevo)`
 - **PUT/PATCH (Editar):** `await Modelo.update(req.body, { where: { id: req.params.id } }); res.json(actualizado)`
 - **DELETE (Borrar):** `await Modelo.destroy({ where: { id: req.params.id } }); res.json('Borrado')`